Unity Catalog Tables & Delta Lake

> **Document Owner:** Platform Engineering | **Status:** Approved | **Last Updated:** February 2026

Overview

Unity Catalog is the unified governance layer for all data and AI assets, and Delta Lake is the mandatory storage format. This document defines standards for table types, storage, properties, documentation, and Delta Lake operational best practices.

Golden Rules

ID	Rule	Severity	WAF Pillar
DL-01	All tables in Unity Catalog (no HMS)	Critical	Governance
DL-02	Managed tables are the default	Critical	Governance
DL-03	External tables require approval	Required	Governance
DL-04	All tables use Delta Lake format	Critical	Interop
DL-05	CLUSTER BY AUTO on all tables	Critical	Performance
DL-06	Standard TBLPROPERTIES required	Required	OpEx
DL-07	LLM-friendly COMMENTs required	Required	Governance
DL-08	Never use Spark caching with Delta tables	Required	Performance
DL-09	Never manually modify Delta data files	Critical	Reliability
DL-10	Remove legacy Delta configurations on upgrade	Recommended	Performance
DL-11	Enable UniForm for cross-engine interoperability	Recommended	Interop
DL-12	Run ANALYZE TABLE on performance-critical tables	Recommended	Performance

---

Managed Tables (Default)

Managed tables are the recommended table type, offering automatic optimization and lower costs.

Why Managed Tables?

Benefit	Description
Reduced costs	AI-driven optimization
Faster queries	Automatic maintenance
Secure access	Open APIs for external tools

Benefit	Description
Auto cleanup	Files deleted 8 days after DROP

Creating Managed Tables

```
-- Managed table (NO LOCATION clause)
CREATE TABLE catalog.schema.my_table (
    id BIGINT,
    name STRING,
    created_at TIMESTAMP
)
USING DELTA
CLUSTER BY AUTO;
```

External Tables (Requires Approval)

External tables point to your own cloud storage. Use only when data must persist after DROP.

```
-- External table (requires External Location grant)
CREATE TABLE catalog.schema.external_table (...)
USING DELTA
LOCATION 's3://my-bucket/data/external_table';
```

Scenario	Approval
Data sharing with external systems	Platform Architect
Regulatory data retention	Compliance + Platform
Legacy migration	Team Lead (temporary)

Automatic Liquid Clustering

Every table should use automatic clustering:

```
CREATE TABLE catalog.schema.my_table (...)
USING DELTA
CLUSTER BY AUTO;

-- Enable on existing table
ALTER TABLE catalog.schema.my_table CLUSTER BY AUTO;
```

Why CLUSTER BY AUTO?

Manual Clustering	Automatic (AUTO)
Requires analysis	Self-tuning
Static over time	Adapts to queries
May become stale	Always optimized

Anti-Patterns

```
-- Wrong: Manual column specification
-- CLUSTER BY (sale_date, region)

-- Wrong: Legacy Z-ORDER
-- OPTIMIZE table ZORDER BY (column);
```

Required Table Properties

```
CREATE TABLE catalog.schema.fact_orders (...)
USING DELTA
CLUSTER BY AUTO
TBLPROPERTIES (
    -- Performance
    'delta.enableChangeDataFeed' = 'true',
    'delta.enableRowTracking' = 'true',
    'delta.enableDeletionVectors' = 'true',
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true',

    -- Governance
    'layer' = 'gold',
    'domain' = 'sales',
    'entity_type' = 'fact',
    'contains_pii' = 'false',
    'data_classification' = 'internal'
);
```

Documentation Standards

Table Comments

```
COMMENT ON TABLE gold.fact_orders IS
'Daily order facts aggregated by customer and product.
Business: Primary source for revenue reporting.
Grain: One row per order_id.
Technical: Source is Silver layer order stream.';
```

Column Comments

```
COMMENT ON COLUMN gold.fact_orders.total_amount IS
'Total order amount in USD after discounts.
Business: Primary revenue metric.
Technical: quantity * unit_price * (1 - discount_pct).';
```

Primary/Foreign Keys

```
-- Add PK (informational, not enforced)
ALTER TABLE gold.dim_customer
ADD CONSTRAINT pk_dim_customer
PRIMARY KEY (customer_key) NOT ENFORCED;

-- Add FK
ALTER TABLE gold.fact_orders
ADD CONSTRAINT fk_orders_customer
FOREIGN KEY (customer_key)
REFERENCES gold.dim_customer(customer_key) NOT ENFORCED;
```

Schema Management

Enable Predictive Optimization

Predictive optimization automatically runs OPTIMIZE and VACUUM — no manual scheduling needed.

```
-- Enable at schema level (recommended)
ALTER SCHEMA catalog.gold ENABLE PREDICTIVE OPTIMIZATION;

-- Enable at catalog level (broader scope)
ALTER CATALOG my_catalog ENABLE PREDICTIVE OPTIMIZATION;

-- Verify status
DESCRIBE SCHEMA EXTENDED catalog.gold;
```

Removes Need For

- Manual OPTIMIZE jobs
- Manual VACUUM scheduling
- File size tuning configurations

Schema Properties (Asset Bundle)

```yaml
resources:
  schemas:
    gold_schema:
      name: gold
      catalog_name: ${var.catalog}
      comment: "Gold layer - business aggregates"
      properties:
        layer: "gold"
        delta.autoOptimize.optimizeWrite: "true"
```

Access Control

```sql
-- Schema-level grants (preferred)
GRANT USE SCHEMA ON SCHEMA catalog.gold TO `analysts`;
GRANT SELECT ON SCHEMA catalog.gold TO `analysts`;

-- Function grants
GRANT EXECUTE ON FUNCTION catalog.gold.get_daily_orders TO `analysts`;
```

DL-08: No Spark Caching

Never use `.cache()` or `.persist()` on Delta tables.

Why It Matters

- Caching bypasses Delta's data skipping
- Cached data may become stale after updates
- Consumes executor memory unnecessarily
- Delta automatically optimizes repeated reads

Anti-Pattern

```python
# WRONG: Caching Delta table
df = spark.table("catalog.schema.my_table")
df.cache()  # Loses data skipping!
df.filter("date = '2025-01-01'").count()
```

Correct Pattern

```python
# CORRECT: Let Delta handle optimization
df = spark.table("catalog.schema.my_table")
df.filter("date = '2025-01-01'").count()  # Uses data skipping
```

DL-09: No Manual File Modifications

Never directly modify, add, or delete Parquet files in a Delta table location.

Why It Matters

- Delta uses transaction log for atomic commits
- Manual changes bypass the log, causing inconsistency
- Results in lost data, duplicates, or corruption
- Queries may fail with missing file errors

Prohibited Operations

```
# Never do this
aws s3 rm s3://bucket/delta-table/part-00001.parquet
aws s3 cp data.parquet s3://bucket/delta-table/
```

Correct Operations

```
-- Use Delta operations
DELETE FROM table WHERE condition;
INSERT INTO table SELECT ...;
UPDATE table SET column = value WHERE condition;
VACUUM table RETAIN 168 HOURS;
```

DL-10: Remove Legacy Configurations

Remove explicit legacy Delta configurations when upgrading Databricks Runtime. Let the platform use optimized defaults.

Configurations to Review

Configuration	Action
`delta.autoOptimize.optimizeWrite`	Remove if predictive optimization enabled
`delta.autoOptimize.autoCompact`	Remove if predictive optimization enabled
`spark.sql.shuffle.partitions`	Let AQE auto-tune
`delta.targetFileSize`	Remove (automatic tuning)

Migration

```sql
-- Check current properties
SHOW TBLPROPERTIES catalog.schema.my_table;

-- Remove legacy properties
ALTER TABLE catalog.schema.my_table
UNSET TBLPROPERTIES ('delta.targetFileSize');
```

DL-11: UniForm for Interoperability

Enable Delta UniForm on tables that need to be read by non-Databricks engines (Spark, Trino, Presto, BigQuery, Snowflake). This generates Iceberg-compatible metadata alongside Delta metadata.

Implementation

```sql
-- Enable UniForm on new table
CREATE TABLE catalog.schema.shared_table (...)
USING DELTA
CLUSTER BY AUTO
TBLPROPERTIES (
    'delta.universalFormat.enabledFormats' = 'iceberg'
);

-- Enable on existing table
ALTER TABLE catalog.schema.existing_table
SET TBLPROPERTIES ('delta.universalFormat.enabledFormats' = 'iceberg');
```

When to Enable UniForm

Scenario	Enable UniForm?
Tables consumed only by Databricks	No (unnecessary overhead)
Tables shared via Delta Sharing	No (Delta Sharing handles cross-platform)
Tables read by Snowflake, BigQuery, Trino	Yes
Tables in open data lakehouse with multiple engines	Yes

DL-12: Run ANALYZE TABLE

Run ANALYZE TABLE on performance-critical tables to collect statistics that enable better query optimization.

Implementation

```sql
-- Collect statistics for a specific table
ANALYZE TABLE catalog.schema.fact_orders COMPUTE STATISTICS;

-- Collect statistics for specific columns
ANALYZE TABLE catalog.schema.fact_orders
COMPUTE STATISTICS FOR COLUMNS order_date, customer_id, total_amount;

-- Verify statistics are collected
DESCRIBE EXTENDED catalog.schema.fact_orders;
```

MERGE Performance Tips

Reduce Search Space

```sql
-- Faster: Constrained MERGE
MERGE INTO events AS target
USING updates AS source
ON target.id = source.id
   AND target.event_date = current_date()  -- Partition filter
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

-- Slower: Full table scan
MERGE INTO events AS target
USING updates AS source
ON target.id = source.id  -- Scans entire table
...
```

Operations You Don't Need

Delta handles these automatically — do not run them manually:

Operation	Why Not Needed
`REFRESH TABLE`	Delta always returns current data
`ALTER TABLE ADD PARTITION`	Partitions tracked automatically
`MSCK REPAIR TABLE`	Not applicable to Delta
Direct partition reads	Use WHERE clause instead

Validation Checklist

Table Governance

- [ ] Created in Unity Catalog (not HMS)

- [ ] Uses DELTA format
- [ ] CLUSTER BY AUTO applied
- [ ] Required TBLPROPERTIES set
- [ ] Table COMMENT present
- [ ] All columns have COMMENT
- [ ] PII columns tagged

Gold Layer

- [ ] PK constraint added
- [ ] FK constraints added (after all PKs exist)

Delta Lake Operations

- [ ] No `.cache()` or `.persist()` on Delta tables in code
- [ ] No direct file system operations on Delta paths
- [ ] Legacy configurations removed after runtime upgrades
- [ ] MERGE operations include partition filters where possible
- [ ] Predictive optimization enabled at schema/catalog level

---

Quick Reference

```sql
-- Enable predictive optimization
ALTER SCHEMA catalog.schema ENABLE PREDICTIVE OPTIMIZATION;

-- Create table with auto clustering
CREATE TABLE catalog.schema.my_table (...)
USING DELTA CLUSTER BY AUTO;

-- Enable on existing table
ALTER TABLE catalog.schema.my_table CLUSTER BY AUTO;

-- Remove legacy config
ALTER TABLE catalog.schema.my_table
UNSET TBLPROPERTIES ('legacy.config');
```

Related Documents

- Serverless Compute
- Data Governance
- Reliability & DR

References

- Unity Catalog Managed Tables
- Liquid Clustering

- Unity Catalog
- Delta Lake Best Practices
- Predictive Optimization
- Delta UniForm